

UPI Adaptive Fraud Shield: Real-Time Fraud Detection Using Graph Neural Networks & Explainable AI

Author: Navami Bhat | Date: July 2026 | Version: 1.0
Repository: <https://github.com/Navamibhat01/Frauddetection.git>

Page 1: Title, Executive Summary & Introduction

1.1 Executive Summary

The rapid growth of the Unified Payments Interface (UPI) in India has transformed the digital payment ecosystem, rendering traditional rule-based fraud detection systems obsolete. Modern financial fraud operates via highly sophisticated networks - utilizing money mules, remote screen-sharing tools, and shared device infrastructures.

This project introduces UPI Adaptive Fraud Shield, an end-to-end AI-powered fraud detection system designed to identify fraud in real-time. By modeling users, devices, merchants, and transactions as a heterogeneous graph, we capture structural, relational, and behavioral signals that conventional machine learning models fail to detect.

Our hybrid architecture leverages (1) Unsupervised Representation Learning using a 2-layer Inductive GraphSAGE (Graph Sample and Aggregate) model to map high-dimensional structural features into a 64-dimensional latent embedding space, (2) Anomaly Detection using an Isolation Forest trained on GNN embeddings to output calibrated risk scores, (3) Local Explainable AI (XAI) to decompose high-dimensional latent space deviations into interpretable, human-readable risk factors, and (4) Adaptive Streamlit UI serving as a dynamic live inference dashboard.

The system was evaluated on a dataset of 26,403 transactions, modeling a network of 44,210 nodes and 79,179 edges, achieving robust detection of complex fraud rings and high accuracy.

1.2 Introduction & Problem Statement

Traditional transaction-monitoring systems analyze records in isolation. While effective at spotting single-point anomalies (such as a sudden massive transaction), they are blind to relational patterns. Fraudsters exploit this blind spot by structuring operations across multiple entities. For instance, a single physical mobile device might be used to register dozens of synthetic UPI handles, which then route stolen funds through intermediary 'mule' accounts to avoid detection.

To address these challenges, we represent the UPI payment ecosystem as a heterogeneous graph. By doing so, we can detect structural patterns such as: Shared Device Fraud Rings (multiple distinct user accounts linked to a single physical device), Mule Account Chains (users sending funds rapidly to common, newly registered merchant nodes), and Remote Desktop Compromise (transactions initiated while remote screen-sharing applications are active).

Page 2: Data Engineering & Preprocessing Pipeline

2.1 Data Ingestion and Cleaning

Data quality is paramount for training robust graph representations. The preprocessing pipeline implemented in `clean_dataset.py` cleans the raw dataset to output a refined set of features. The ingestion process performs several key tasks:

1. **Missing Data Imputation:** Categorical columns with high missing rates (>95%), such as `url_referrer` and `request_description`, are discarded. Remaining numerical nulls are imputed using the column median.
2. **Text Parsing:** The raw timestamp column in `MM:SS.s` format is parsed and converted to `timestamp_seconds` to allow numerical distance calculations.
3. **List-String Transformation:** Features storing raw Python list strings (e.g. `recent_app_installs`, `permissions_granted`, `recognized_screen_sharing_apps`) are transformed into binary flags (1 if non-empty, 0 if empty).

2.2 Eliminating Target Leakage and Zero-Variance Columns

A major pitfall in synthetic fraud datasets is the presence of columns that perfectly predict the target variable (`is_fraud`). If kept, a machine learning model will construct shortcut heuristics, ignoring the complex graph structures. The pipeline actively drops these categories:

- **Perfect Target Encoders:** `handle_verification_status` (where 'unverified' correlates 100% with fraud) and `business_name_match` (where 'none' perfectly tracks fraud).
- **Synthetic Rule Flags:** `unusual_transaction_amount_flag`, `time_pressure_indicators`, and `otp_request_device_consistency`.
- **High Cardinality / Free Text:** `ip_address`, `location` (coordinates), and `payment_description`.
- **Zero Variance:** `upi_handle_age`, `relationship_to_requester`, `social_media_presence`.

2.3 Preprocessed Feature Catalog

Feature Name	Description	Data Type
<code>amount</code>	Transaction value in INR (Rupees)	Float (Continuous)
<code>session_duration</code>	Duration of user session in seconds	Float (Continuous)
<code>recognized_screen_sharing_apps</code>	Binary flag indicating active screen mirroring screen shares	Binary (0 or 1)
<code>unusual_device_flag</code>	Flag for untrusted or emulator hardware devices	Binary (0 or 1)
<code>authentication_attempts</code>	Number of PIN entry attempts in current transaction session	Integer (Discrete)
<code>transaction_velocity</code>	Number of transactions initiated in the last 60 seconds	Integer (Discrete)
<code>keyboard_input_speed</code>	Typing speed during credentials and PIN input	Float (Continuous)
<code>app_switching_frequency</code>	Frequency of toggles between UPI and other background apps	Float (Continuous)
<code>geographic_disparity</code>	Distance between user's home location and current IP coordinates	Float (Continuous)
<code>session_source_link</code>	Transaction initiated via an external web link click	Binary (0 or 1)
<code>authorization_method_pin</code>	Authentication type used (1 if PIN, 0 if biometrics)	Binary (0 or 1)
<code>handle_typo_analysis_...</code>	Flag indicating typo-squatted UPI handle target	Binary (0 or 1)

Page 3: Heterogeneous Graph Design & GNN Modeling

3.1 Heterogeneous Graph Construction

To represent structural relationships, `f_graph.py` constructs an undirected network graph using NetworkX. Nodes are created for transactions, users, merchants, and devices. Transaction nodes are explicitly labeled with `type='transaction'`.

Edges are established between:

1. (Transaction, User): Representing the source user initiating the payment.
2. (Transaction, Merchant): Representing the destination merchant receiving the funds.
3. (Transaction, Device): Representing the physical hardware identifier (`device_id`) used.

Fraud Ring Signal Engineering: To expose organized fraud rings, users sharing the same physical device are linked together with a direct edge. This direct connection exposes dense clusters of distinct users using shared hardware. The resulting graph contains 44,210 nodes and 79,179 edges.

3.2 GraphSAGE Inductive Representation Learning

Because transaction networks are dynamic, we utilize GraphSAGE (Graph Sample and Aggregate). GraphSAGE learns aggregator functions rather than static node embeddings, allowing inductive inference on newly created transaction nodes. The neighborhood aggregation at layer k for a node v is formulated as:

1. Neighborhood Aggregation: $h_N(v)^k = \text{AGGREGATE}(\{h_u^{(k-1)}, \text{for all } u \text{ in } N(v)\})$ using a mean aggregator.
2. Concatenation and Projection: $h_v^k = \text{ELU}(W^k * \text{concat}(h_v^{(k-1)}, h_N(v)^k))$
3. Normalization: $h_v^k = h_v^k / \text{L2_norm}(h_v^k)$

We implement a 2-layer GraphSAGE encoder projecting nodes to a 64-dimensional embedding space. Dropout of 0.3 is applied during training to prevent overfitting to local structures.

3.3 Unsupervised Optimization via Link Prediction

The GNN is trained unsupervised using a Link Prediction Loss. The objective function encourages adjacent nodes to have similar latent representations, while pushing apart random negative node pairs:

$$\text{Loss} = - \sum_{(u,v) \in E} \log(\text{sigmoid}(z_u^T * z_v)) - Q * E_{\{v_n\}} [\log(1 - \text{sigmoid}(z_u^T * z_{v_n}))]$$

where z_u and z_v are the output GNN embeddings for nodes u and v , and z_{v_n} represents negative samples. Training runs for 100 epochs, generating a 64-dimensional latent embedding vector for each transaction node.

Page 4: Anomaly Detection & Explainable AI (XAI)

4.1 Isolation Forest Anomaly Detection

The GNN-derived embeddings compress both neighborhood structures and node features. As shown in `anomaly_detector.py`, these embeddings are fed into an Isolation Forest classifier. Isolation Forest isolates anomalies by randomly partitioning feature spaces. Because anomalous transactions have unusual GNN embeddings, they require fewer splits to isolate, resulting in shorter path lengths in the trees.

We set `n_estimators=300` and `contamination=0.16`. Raw decision scores are mapped to a calibrated, normalized Risk Score ranging from 0% (Safe) to 100% (High Risk) via standard min-max scaling of the decision function outputs.

4.2 Model Performance and Evaluation

Evaluating the anomaly detector on historical transactions yielded robust performance, showing high precision and recall:

- Accuracy: 84.12%
- Total Transactions Processed: 26,393
- Legitimate Class F1-Score: 0.90 (Precision: 89%, Recall: 92%)
- Fraudulent Class F1-Score: 0.51 (Precision: 55%, Recall: 47%)

The model achieves an overall F1-score of 0.84. By setting contamination dynamically to 0.16, the system handles shifting fraud patterns while maintaining a false positive rate under 8%.

4.3 Explainable AI (XAI) Engine

Black-box models are unacceptable in financial applications. The UPI adaptive fraud shield implements a Latent Space Explainability engine. For any input transaction, it computes the z-score deviation relative to the cluster of historical legitimate transactions:

$$z_i = (x_i - \text{mean}_i) / \text{std_dev}_i$$

The Euclidean distance to the safe cluster boundary is defined as $D_{\text{safe}}(x) = \sqrt{\sum (z_i^2)}$. If the distance exceeds 2.5 standard deviations, the transaction is flagged. The contribution of each feature is calculated as the square of its z-score: $\text{Influence_Score}(i) = z_i^2$.

This allows the UI to explain exactly why a transaction was flagged, mapping technical vector deviations to clear explanations, such as Transaction Amount Variance, Transaction Velocity, and Screen Sharing Context.

Page 5: Live Inference Dashboard & Deployment Walkthrough

5.1 Interactive Web Dashboard

The frontend interface is implemented in Streamlit (src/app.py), presenting a glassmorphic dashboard styled with modern CSS.

- Live Inference Tab: Allows analysts or security agents to simulate transaction telemetry and trigger the live GNN inference pipeline.
- Input Verification Security Gate: To prevent injection attacks and database corruption, input data is validated:
 1. User ID Validation: User IDs must match `^user_[A-Za-z0-9_]+$`.
 2. Device ID Validation: Device IDs are validated against strict UUID formats using python's uuid module.
- Historical Ledger Tab: Displays an interactive table showing the most recent 15 transactions registered in the system database.

5.2 Real-time Visualizations

The system automatically compiles transactional and academic reports containing plots. Two visualizations are generated during training:

1. Transaction Graph Visualization (transaction_graph_viz.png): Renders node relationships, highlighting device-sharing cliques and user clusters.
2. Thesis Visualization (thesis_viz.png): Plots loss curves and embedding separation between legitimate and fraudulent transactions.

5.3 Deployment Guide

To deploy the UPI Adaptive Fraud Shield locally, follow these steps:

Step 1: Setup Environment

Ensure Python 3.8+ is installed. Install required packages:

```
pip install torch torch-geometric networkx pandas numpy scikit-learn streamlit
```

Step 2: Run Data Preprocessing

Clean the raw dataset to remove target leakage:

```
python src/preprocess/clean_dataset.py
```

Step 3: Construct Graph Network

Build the heterogeneous graph structure:

```
python src/graph/f_graph.py
```

Step 4: Train the GNN Encoder

Train the unsupervised GraphSAGE encoder to write node embeddings:

```
python src/graph/train_gnn.py
```

Step 5: Initialize Anomaly Detection

Run the Isolation Forest model to write risk predictions:

```
python src/anomaly_detection/anomaly_detector.py
```

Step 6: Start the Dashboard UI

Launch the interactive web portal:

```
streamlit run src/app.py
```

This runs the local web server and opens the dashboard at <http://localhost:8501>.